

Unit 1. Language Preliminaries

8 Hrs.

Introduction to .Net framework, Compilation and execution of .Net applications, Basic Languages constructs, Constructor, Properties, Arrays and String, Indexers, Inheritance, use of “base” keyword, Method hiding and overriding, applying polymorphism in code extensibility, structs and enums, abstract class sealed class, interface, Delegate and Events, Partial class, Collections, Generics, File IO, LINQ (Language Integrated Query) Fundamentals: Lambda Expressions, Try statements and Exceptions, Attributes: Attribute Classes, Named and Positional Attribute Parameters, Attribute Targets, Specifying Multiple Attributes, Asynchronous Programming: Principle of Asynchrony, Async/Await patterns in C#

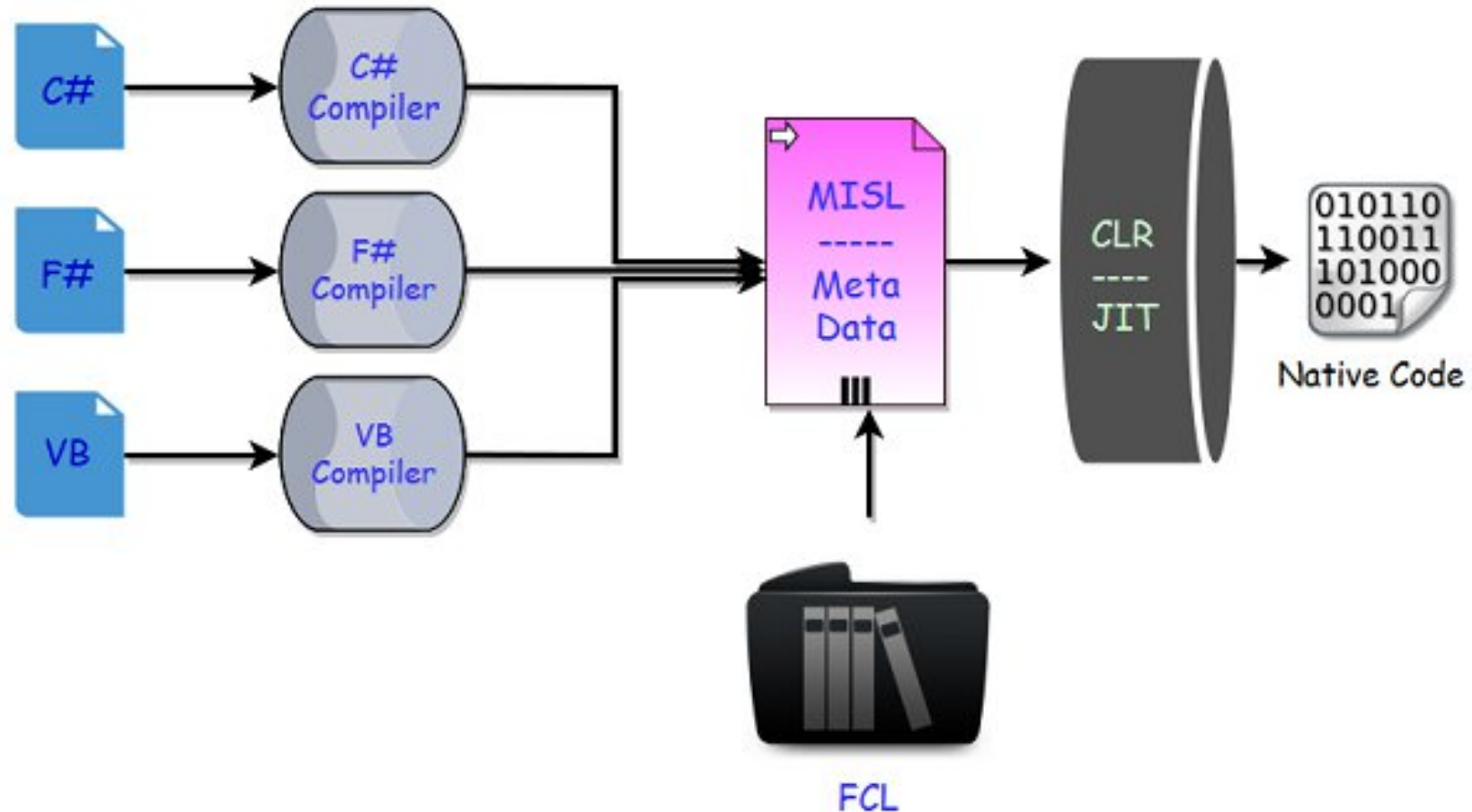
By: Yuba Raj Devkota

CSIT 6th Sem

NCCS

Introduction to .NET Framework

The .NET Framework is a software development platform developed by Microsoft for building and running applications. It provides a common environment for developing desktop, web, mobile, and enterprise applications.



Main Components of .NET Framework

a) Common Language Runtime (CLR)

CLR is the execution engine of .NET. It manages:

- Memory management
- Exception handling
- Security
- Garbage collection
- Thread management

Example:

```
</> C#  
  
int a = 10;  
int b = 20;  
Console.WriteLine(a + b);
```

CLR manages the execution of this code automatically.

b) Framework Class Library (FCL)

A collection of reusable classes, interfaces, and methods.

Example:

```
</> C#  
  
using System;  
  
Console.WriteLine("Hello World");
```

`Console` class comes from the FCL.

c) Common Type System (CTS)

CTS defines how data types are declared and used.

Example:

```
</> C#  
  
int age = 25;  
string name = "Ram";
```

All .NET languages follow the same type system.

d) Common Language Specification (CLS)

CLS provides rules that all .NET languages should follow for interoperability.

Example:

A class written in C# can be used in VB.NET because both follow CLS.

Advantages of .NET Framework

- 1. Language Independence*
- 2. Platform for Rapid Development*
- 3. Automatic Memory Management*
- 4. High Security*
- 5. Large Class Library*
- 6. Easy Integration with Databases and Web Services*

Compilation and Execution of .NET Applications

Unlike C/C++, .NET programs are not compiled directly into machine code.

Steps of Compilation

Step 1: Write Source Code

Example:

```
</> C#  
  
using System;  
  
class Program  
{  
    static void Main()  
    {  
        Console.WriteLine("Hello .NET");  
    }  
}
```

Step 2: Compilation into MSIL/CIL

The C# compiler converts source code into:

Microsoft Intermediate Language (MSIL)

and creates an **Assembly** (.exe or .dll).

Source Code



C# Compiler



MSIL + Metadata

Step 3: JIT Compilation

*When the program runs, CLR's **Just-In-Time (JIT) Compiler** converts MSIL into machine code.*

MSIL



JIT Compiler



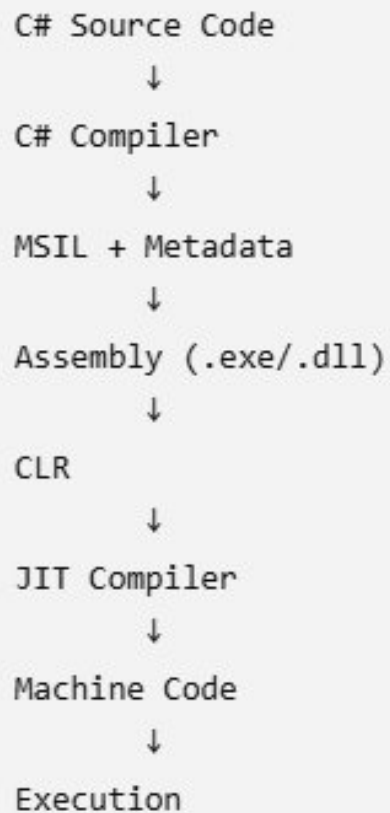
Machine Code

Step 4: Execution by CLR

CLR executes the machine code and provides services like:

- Memory management
- Security
- Exception handling

Complete Flow Diagram



Basic Language Constructs in C#

a) Variables

Variables store data.

`</> C#`

```
int age = 20;  
string name = "Hari";  
double salary = 25000.50;
```

b) Data Types

Data Type	Example
int	10
float	12.5f
double	25.75
char	'A'
string	"Hello"
bool	true

```
int marks = 80;  
bool pass = true;
```


c) Input and Output

Output

```
</> C#  
  
Console.WriteLine("Welcome");
```

Input

```
</> C#  
  
string name = Console.ReadLine();
```

Example:

```
</> C#  
  
Console.Write("Enter Name: ");  
string name = Console.ReadLine();  
Console.WriteLine("Hello " + name);
```

d) Operators

Arithmetic Operators

```
</> C#  
  
int a = 10, b = 5;  
  
Console.WriteLine(a+b);  
Console.WriteLine(a-b);  
Console.WriteLine(a*b);  
Console.WriteLine(a/b);
```

Output

```
15  
5  
50  
2
```

e) Conditional Statements

if Statement

```
</> C#  
  
int age = 18;  
  
if(age >= 18)  
{  
    Console.WriteLine("Eligible to Vote");  
}
```

if-else Statement

```
</> C#  
  
int marks = 45;  
  
if(marks >= 40)  
{  
    Console.WriteLine("Pass");  
}  
else  
{  
    Console.WriteLine("Fail");  
}
```


Summary

- **.NET Framework:** Development platform developed by Microsoft.
- **CLR:** Executes .NET programs and manages memory.
- **FCL:** Collection of reusable classes and libraries.
- **CTS:** Defines common data types for all .NET languages.
- **CLS:** Rules for language interoperability.
- **MSIL/CIL:** Intermediate code generated after compilation.
- **JIT Compiler:** Converts MSIL into machine code at runtime.
- **Assembly:** Compiled .NET file (.exe or .dll).
- **Basic Constructs:** Variables, Data Types, Operators, Conditions, Loops, Arrays, Methods.

Constructor, Properties, Arrays and String

Constructor in C#

A constructor is a special method that is automatically called when an object is created. It is used to initialize object data.

Output

Constructor Called

Example: Default Constructor

```
</> C#  
  
using System;  
  
class Student  
{  
    public Student()  
    {  
        Console.WriteLine("Constructor Called");  
    }  
}  
  
class Program  
{  
    static void Main()  
    {  
        Student s = new Student();  
    }  
}
```

Example: Parameterized Constructor

```
</> C#  
  
using System;  
  
class Student  
{  
    string name;  
  
    public Student(string n)  
    {  
        name = n;  
    }  
  
    public void Display()  
    {  
        Console.WriteLine("Name = " + name);  
    }  
}
```

```
class Program  
{  
    static void Main()  
    {  
        Student s = new Student("Ram");  
        s.Display();  
    }  
}
```

Output

Name = Ram

```
using System;
```

```
class Student
```

```
{
```

```
    private string name;
```

```
    public string Name
```

```
    {
```

```
        get { return name; }
```

```
        set { name = value; }
```

```
    }
```

```
}
```

```
class Program
```

```
{
```

```
    static void Main()
```

```
    {
```

```
        Student s = new Student();
```

```
        s.Name = "Hari";    // set
```

```
        Console.WriteLine(s.Name); // get
```

```
    }
```

```
}
```

Properties in C#

Properties provide controlled access to class fields using get and set.

Output

Hari

Why Properties?

- Encapsulation (Data Hiding)
- Controlled access to data
- Validation can be added easily

Arrays in C#

An array stores multiple values of the same data type in a single variable.

Example 1: Integer Array

```
</> C#  
  
using System;  
  
class Program  
{  
    static void Main()  
    {  
        int[] marks = { 70, 80, 90 };  
  
        Console.WriteLine(marks[0]);  
        Console.WriteLine(marks[1]);  
        Console.WriteLine(marks[2]);  
    }  
}
```

Output

```
70  
80  
90
```

Example 2: Using Loop with Array

```
</> C#  
  
using System;  
  
class Program  
{  
    static void Main()  
    {  
        int[] numbers = { 10, 20, 30, 40, 50 };  
  
        for(int i = 0; i < numbers.Length; i++)  
        {  
            Console.WriteLine(numbers[i]);  
        }  
    }  
}
```

Output

```
10  
20  
30  
40  
50
```

Example 3: Sum of Array Elements

```
</> C#  
  
using System;  
  
class Program  
{  
    static void Main()  
    {  
        int[] num = {10,20,30};  
  
        int sum = 0;  
  
        foreach(int n in num)  
        {  
            sum += n;  
        }  
  
        Console.WriteLine("Sum = " + sum);  
    }  
}
```

Output

```
Sum = 60
```

Strings in C#: A string is a sequence of characters enclosed within double quotes.

Example 1: String Declaration

```
</> C#  
  
using System;  
  
class Program  
{  
    static void Main()  
    {  
        string name = "Ram";  
  
        Console.WriteLine(name);  
    }  
}
```

Output

Ram

Example 2: String Concatenation

```
</> C#  
  
using System;  
  
class Program  
{  
    static void Main()  
    {  
        string firstName = "Ram";  
        string lastName = "Sharma";  
  
        string fullName = firstName + " " + lastName;  
  
        Console.WriteLine(fullName);  
    }  
}
```

Output

Ram Sharma

Example 3: String Length

```
</> C#  
  
using System;  
  
class Program  
{  
    static void Main()  
    {  
        string text = "Kathmandu";  
  
        Console.WriteLine(text.Length);  
    }  
}
```

Output

9

Example 4: Convert to Uppercase

```
</> C#  
  
using System;  
  
class Program  
{  
    static void Main()  
    {  
        string city = "kathmandu";  
  
        Console.WriteLine(city.ToUpper());  
    }  
}
```

Output

KATHMANDU

Example 5: String Comparison

```
</> C#  
  
using System;  
  
class Program  
{  
    static void Main()  
    {  
        string s1 = "Hello";  
        string s2 = "Hello";  
  
        Console.WriteLine(s1 == s2);  
    }  
}
```

Output

True

Summary

Topic	Definition
Constructor	Special method called automatically when an object is created.
Default Constructor	Constructor without parameters.
Parameterized Constructor	Constructor with parameters.
Property	Provides controlled access to class fields using get and set.
Array	Collection of same type elements stored in contiguous memory locations.
String	Sequence of characters enclosed in double quotes.
Common String Functions	Length, ToUpper(), ToLower(), Compare(), Replace()

Classworks

Write a C# program to input a name and display:

- *The name*
- *Length of the name*
- *Name in uppercase*

```
using System;

class Program
{
    static void Main()
    {
        Console.Write("Enter Name: ");
        string name = Console.ReadLine();

        Console.WriteLine("Name: " + name);
        Console.WriteLine("Length: " + name.Length);
        Console.WriteLine("Uppercase: " + name.ToUpper());
    }
}
```

Create a string array containing 5 city names and display only those cities whose names start with the letter 'K'.

```
using System;

class Program
{
    static void Main()
    {
        string[] cities = {"Kathmandu", "Pokhara", "Kirtipur", "Butwal", "Kavre"};

        foreach(string city in cities)
        {
            if(city.StartsWith("K"))
            {
                Console.WriteLine(city);
            }
        }
    }
}
```

Indexers

An array is a data structure that stores elements directly, whereas an indexer allows an object to be accessed like an array while providing controlled access through `get` and `set` methods.

Array

Stores data directly.

Declared with `[]`.

No custom logic when reading/writing.

Example: `int[] a = {1,2,3};`

Indexer

Provides controlled access to data inside an object.

Declared using `this[]`.

Can include validation and custom logic in `get` and `set`.

Example: `student[0] = "Ram";`

Here, `s[0]` actually calls:

```
</> C#  
  
get { ... }  
set { ... }
```

behind the scenes, so the class can control how data is stored or retrieved.

Array

```
</> C#
```

```
int[] marks = {80, 90, 85};  
Console.WriteLine(marks[0]);
```

Here, `marks[0]` directly accesses memory.

Indexer

```
</> C#
```

```
Student s = new Student();  
s[0] = "Ram";  
Console.WriteLine(s[0]);
```

```
using System;

class Student
{
    private string[] names = new string[3];

    public string this[int index]
    {
        get { return names[index]; }
        set { names[index] = value; }
    }
}
```

```
class Program
{
    static void Main()
    {
        Student s = new Student();

        s[0] = "Ram";
        s[1] = "Hari";
        s[2] = "Sita";

        Console.WriteLine(s[0]);
        Console.WriteLine(s[1]);
        Console.WriteLine(s[2]);
    }
}
```

Output

Ram
Hari
Sita

Inheritance

Inheritance allows a child class to acquire properties and methods of a parent class.

```
using System;

class Animal
{
    public void Eat()
    {
        Console.WriteLine("Animal is eating");
    }
}

class Dog : Animal
{
    public void Bark()
    {
        Console.WriteLine("Dog is barking");
    }
}
```

```
class Program
{
    static void Main()
    {
        Dog d = new Dog();

        d.Eat();
        d.Bark();
    }
}
```

Output

```
Animal is eating
Dog is barking
```

Use of base keyword:

The base keyword is used to access members of the parent class from a derived class.

```
using System;

class Animal
{
    public void Sound()
    {
        Console.WriteLine("Animal makes sound");
    }
}

class Dog : Animal
{
    public void Sound()
    {
        base.Sound();
        Console.WriteLine("Dog barks");
    }
}
```

```
class Program
{
    static void Main()
    {
        Dog d = new Dog();
        d.Sound();
    }
}
```

Output

```
Animal makes sound
Dog barks
```

Method Hiding

Method hiding occurs when a derived class defines a method with the same name as the base class using the new keyword

```
using System;

class Parent
{
    public void Show()
    {
        Console.WriteLine("Parent Class");
    }
}

class Child : Parent
{
    public new void Show()
    {
        Console.WriteLine("Child Class");
    }
}
```

```
class Program
{
    static void Main()
    {
        Parent p = new Child();

        p.Show();
    }
}
```

Output

Parent Class

The method called depends on the reference type {parent}

Method Overriding

Method overriding allows a child class to provide a different implementation of a method defined in the parent class using *virtual* and *override*.

```
using System;

class Parent
{
    public virtual void Show()
    {
        Console.WriteLine("Parent Class");
    }
}

class Child : Parent
{
    public override void Show()
    {
        Console.WriteLine("Child Class");
    }
}
```

```
class Program
{
    static void Main()
    {
        Parent p = new Child();

        p.Show();
    }
}
```

the method called depends on the actual object {child} not the reference type.

Output

Child Class

Applying Polymorphism for Code Extensibility

Polymorphism allows one interface to represent many forms. It makes programs easier to extend without modifying existing code.

```
using System;

class Employee
{
    public virtual double CalculateSalary()
    {
        return 0;
    }
}

class FullTimeEmployee : Employee
{
    public override double CalculateSalary()
    {
        return 50000;
    }
}
```

```
class PartTimeEmployee : Employee
{
    public override double CalculateSalary()
    {
        return 20000;
    }
}

class Program
{
    static void Main()
    {
        Employee[] employees =
        {
            new FullTimeEmployee(),
            new PartTimeEmployee()
        };

        foreach(Employee emp in employees)
        {
            Console.WriteLine(emp.CalculateSalary());
        }
    }
}
```

Output

```
50000
20000
```

Why is it Extensible?

Suppose a new employee type is added:

```
</> C#  
  
class ContractEmployee : Employee  
{  
    public override double CalculateSalary()  
    {  
        return 30000;  
    }  
}
```

No changes are required in the existing loop:

```
</> C#  
  
Employee emp = new ContractEmployee();  
Console.WriteLine(emp.CalculateSalary());
```

Output

30000

New classes can be added without modifying existing code, making the application flexible, maintainable, and extensible.

- **Indexer:** Allows object access using array syntax (`obj[index]`).
- **Inheritance:** Child class acquires members of parent class.
- **base:** Refers to members of the parent class.
- **Method Hiding:** Uses `new` ; depends on reference type.
- **Method Overriding:** Uses `virtual` and `override` ; depends on object type.
- **Polymorphism:** One interface, many forms; improves code extensibility and maintenance.

Problem

Create a class named `Student` that:

- Stores marks of 5 subjects in an array.
- Uses an **Indexer** to insert and retrieve marks.
- Displays all marks and calculates the total.

Sample Output

Marks :

80

75

90

85

70

Total = 400

```
using System;
```

```
class Student
{
    private int[] marks = new int[5];

    public int this[int index]
    {
        get { return marks[index]; }
        set { marks[index] = value; }
    }
}
```

Classwork : Student Result Using Indexer

```
class Program
{
    static void Main()
    {
        Student s = new Student();

        s[0] = 80;
        s[1] = 75;
        s[2] = 90;
        s[3] = 85;
        s[4] = 70;

        int total = 0;

        Console.WriteLine("Marks:");
        for(int i=0;i<5;i++)
        {
            Console.WriteLine(s[i]);
            total += s[i];
        }

        Console.WriteLine("\nTotal = " + total);
    }
}
```

C# Structs and Enums

Structs

A struct (structure) is a value type used to store small groups of related data. It is similar to a class but is stored on the stack (in most cases) and is copied by value.

```
using System;

struct Student
{
    public int Id;
    public string Name;

    public void Display()
    {
        Console.WriteLine("ID: " + Id);
        Console.WriteLine("Name: " + Name);
    }
}
```

```
class Program
{
    static void Main()
    {
        Student s;

        s.Id = 101;
        s.Name = "Ram";

        s.Display();
    }
}
```

Output

ID: 101
Name: Ram

```
using System;
```

```
struct Point
```

```
{
```

```
    public int X;
```

```
    public int Y;
```

```
}
```

```
class Program
```

```
{
```

```
    static void Main()
```

```
    {
```

```
        Point p;
```

```
        p.X = 10;
```

```
        p.Y = 20;
```

```
        Console.WriteLine($"({p.X}, {p.Y})");
```

```
    }
```

```
}
```

Struct vs Class

Struct

Value Type

Stored mostly on Stack

Passed by value

Cannot inherit classes

Lightweight

Class

Reference Type

Stored on Heap

Passed by reference

Supports inheritance

More flexible

Output

(10, 20)

Enum (Enumeration)

```
using System;
```

```
enum Day
```

```
{
```

```
    Sunday,
```

```
    Monday,
```

```
    Tuesday,
```

```
    Wednesday,
```

```
    Thursday,
```

```
    Friday,
```

```
    Saturday
```

```
}
```

```
class Program
```

```
{
```

```
    static void Main()
```

```
    {
```

```
        Day today = Day.Friday;
```

```
        Console.WriteLine(today);
```

```
        Console.WriteLine((int)today);
```

```
    }
```

```
}
```

- An *enum* is a user-defined data type that contains a fixed set of named constants.
- Instead of remembering numbers like 0,1,2, we use meaningful names.

Output

Friday

5

(The numbering starts from 0.)

Advantages of Enum

- Improves code readability
 - Avoids magic numbers
 - Makes programs easier to maintain
- Represents fixed choices

Examples:

- Months
- Days
- Traffic Lights
- Employee Status

Abstract Class, Sealed Class and Interface

An abstract class is a class that cannot be instantiated (cannot create objects directly). It is mainly used as a base class to provide common functionality and force derived classes to implement certain methods.

Key Points

- Declared using the `abstract` keyword.
- Cannot create objects directly.
- Can contain:
 - Abstract methods (without implementation)
 - Normal methods (with implementation)
 - Constructors
 - Fields and properties
- A derived class **must implement** all abstract methods.

Syntax

```
</> C#  
  
abstract class Animal  
{  
    public abstract void Sound();  
  
    public void Eat()  
    {  
        Console.WriteLine("Animal is eating.");  
    }  
}
```

```
using System;

abstract class Animal
{
    public abstract void Sound();

    public void Eat()
    {
        Console.WriteLine("Animal is eating.");
    }
}

class Dog : Animal
{
    public override void Sound()
    {
        Console.WriteLine("Dog barks.");
    }
}
```

```
class Program
{
    static void Main()
    {
        Dog d = new Dog();

        d.Sound();
        d.Eat();
    }
}
```

Output

```
Dog barks.
Animal is eating.
```

Why use an Abstract Class?

Suppose every vehicle should have a Start() method, but different vehicles start differently.

```
abstract class Vehicle
{
    public abstract void Start();
}

class Car : Vehicle
{
    public override void Start()
    {
        Console.WriteLine("Car starts with key.");
    }
}

class Bike : Vehicle
{
    public override void Start()
    {
        Console.WriteLine("Bike starts with self-start.");
    }
}
```

This ensures every vehicle has a Start() method.

Sealed Class

A **sealed class** is a class that **cannot be inherited**.

Once a class is declared as `sealed`, no other class can derive from it.

Key Points

- Declared using the `sealed` keyword.
- Cannot be used as a base class.
- Objects can still be created.
- Used when you want to prevent inheritance.

Syntax

`</> C#`

```
sealed class Calculator
{
}
```

Output

30

```
using System;

sealed class Calculator
{
    public int Add(int a, int b)
    {
        return a + b;
    }
}

class Program
{
    static void Main()
    {
        Calculator c = new Calculator();

        Console.WriteLine(c.Add(10, 20));
    }
}
```

Invalid Example

```
</> C#  
  
sealed class Animal  
{  
}  
  
class Dog : Animal  
{  
}
```

Compile-Time Error

Cannot derive from sealed type 'Animal'

Abstract Class vs Sealed Class

Feature	Abstract Class	Sealed Class
Can create object?	 No	 Yes
Can be inherited?	 Yes	 No
Purpose	Base class for inheritance	Prevent inheritance
Keyword	abstract	sealed
Can have abstract methods?	 Yes	 No
Constructor allowed?	 Yes	 Yes

Interface

- *An interface is a contract that defines what a class must do, but not how it does it. Any class that implements an interface must provide implementations for all its members.*
- *Unlike an abstract class, an interface contains only member declarations (traditionally methods, properties, events, and indexers) and no implementation. It is mainly used to achieve abstraction and multiple inheritance in C#.*

Key Points

- Declared using the `interface` keyword.
- Cannot create objects directly.
- Contains only declarations (methods, properties, events, indexers).
- A class implements an interface using the `:` symbol.
- A class can implement **multiple interfaces**.
- Interface members are **public** by default.

Syntax

```
</> C#  
  
interface IAnimal  
{  
    void Sound();  
}
```

A class can implement more than one interface. (Multiple Interface)

```
using System;

interface IAnimal
{
    void Sound();
}

class Dog : IAnimal
{
    public void Sound()
    {
        Console.WriteLine("Dog barks.");
    }
}
```

Output

Dog barks.

```
class Program
{
    static void Main()
    {
        Dog d = new Dog();
        d.Sound();
    }
}
```

```
using System;

interface IFly
{
    void Fly();
}

interface ISwim
{
    void Swim();
}

class Duck : IFly, ISwim
{
    public void Fly()
    {
        Console.WriteLine("Duck can fly.");
    }

    public void Swim()
    {
        Console.WriteLine("Duck can swim.");
    }
}
```

```
class Program
{
    static void Main()
    {
        Duck d = new Duck();

        d.Fly();
        d.Swim();
    }
}
```

Output

Duck can fly.
Duck can swim.

Delegate and Events

Delegates and Events are important features in C# that support event-driven programming. They allow one object to notify another object when something happens.

A delegate is a type-safe function pointer. It can hold the reference (address) of one or more methods having the same signature (same return type and parameters).

Why use Delegates?

- Call methods indirectly.
- Pass methods as parameters.
- Implement callbacks.
- Used in Events.

Syntax

</> C#

```
delegate returnType DelegateName(parameterList);
```

```
using System;

delegate void MyDelegate();

class Program
{
    static void Hello()
    {
        Console.WriteLine("Hello Students!");
    }

    static void Main()
    {
        MyDelegate d = Hello;
        d();           // Calls Hello()
    }
}
```

Output

Hello Students!

Delegate with Parameters

```
using System;

delegate int Calculator(int x, int y);

class Program
{
    static int Add(int a, int b)
    {
        return a + b;
    }

    static void Main()
    {
        Calculator c = Add;

        int result = c(10, 20);

        Console.WriteLine("Sum = " + result);
    }
}
```

Output

Sum = 30

Events

An Event is a mechanism by which one object notifies other objects that something has happened. An event is based on delegates. Examples:

- *Button Click*
- *Mouse Click*
- *Key Press*
- *Timer Tick*
- *Form Load*

Real-life Example

Think of a school bell.

- Bell rings → Event occurs
- Students and teachers hear it and respond.

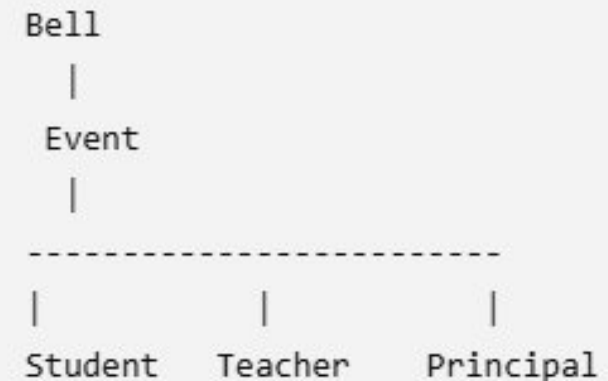
The bell doesn't know who is listening.

Event Syntax

</> C#

```
public delegate void MyDelegate();

public event MyDelegate MyEvent;
```



Delegate vs Event

Feature	Delegate	Event
Purpose	Stores reference to methods	Notifies subscribers when something happens
Based on	Function pointer	Delegate
Can invoke?	Yes	Only inside the declaring class
Supports multiple methods	Yes	Yes
Used for	Callback methods	Event-driven programming
Access	Any code with the delegate can invoke it	Outside code can only subscribe (<code>+=</code>) or unsubscribe (<code>-=</code>)

How do you relate delegates with events?

A simple way to remember the relationship is:

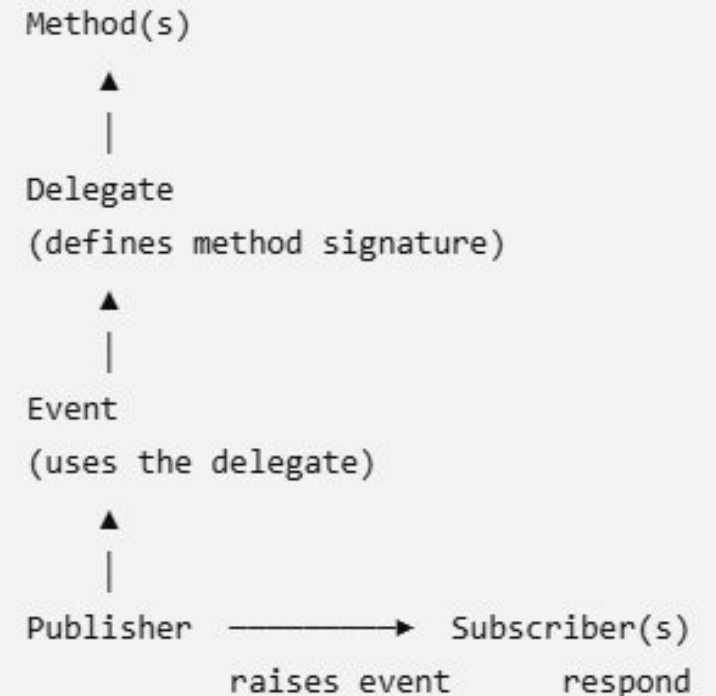
An event is built on a delegate.

A delegate defines *what kind of method* can be called, while an event uses that delegate to notify subscribers when something happens.

Analogy

- **Delegate** → A phone number (it tells you *which person/method* can be contacted).
- **Event** → A phone call (it uses the phone number to notify someone).

Relationship Diagram



```

using System;

// Step 1: Declare a delegate
delegate void Notify();

class Alarm
{
    // Step 2: Declare an event using the delegate
    public event Notify Ring;

    public void Start()
    {
        Console.WriteLine("Alarm Ringing...");
        Ring?.Invoke(); // Raise the event
    }
}

```

```

class Program
{
    static void WakeUp()
    {
        Console.WriteLine("Wake up!");
    }

    static void Main()
    {
        Alarm alarm = new Alarm();

        // Subscribe the method to the event
        alarm.Ring += WakeUp;

        // Raise the event
        alarm.Start();
    }
}

```

Output

```

Alarm Ringing...
Wake up!

```

Delegate = "How to call a method";
Event = "When to call those methods."

Partial class

A **Partial Class** is a class whose definition is **split into two or more files**. The `partial` keyword tells the C# compiler to combine all parts into a **single class** during compilation.

File: StudentInfo.cs

```
</> C#  
  
using System;  
  
public partial class Student  
{  
    public string Name = "Ram";  
  
    public void DisplayName()  
    {  
        Console.WriteLine("Name: " + Name);  
    }  
}
```

File: StudentMarks.cs

```
</> C#  
  
using System;  
  
public partial class Student  
{  
    public int Marks = 85;  
  
    public void DisplayMarks()  
    {  
        Console.WriteLine("Marks: " + Marks);  
    }  
}
```

File: Program.cs

```
</> C#  
  
using System;  
  
class Program  
{  
    static void Main()  
    {  
        Student s = new Student();  
  
        s.DisplayName();  
        s.DisplayMarks();  
    }  
}
```

Output

```
Name: Ram  
Marks: 85
```

Although student is written in two files, the compiler treats it as one class.

What are the needs of Partial and Sealed Class?

Need of Partial Class

Partial classes are useful when:

- **Large classes** can be divided into smaller, manageable files.
- **Multiple developers** can work on different parts of the same class simultaneously.
- **Auto-generated code** (e.g., Windows Forms, ASP.NET designers) is kept separate from manually written code.
- Improves **code organization** and readability.

Need of Sealed Class

Sealed classes are useful when:

- You want to **prevent inheritance**.
- You want to **protect the implementation** from being modified through derived classes.
- You are creating **final or secure classes** that should not be extended.
- It can provide **minor performance optimizations** because the compiler knows the class cannot be overridden.

Examples from .NET include classes such as `String`, which is sealed.

Collections and Generics in C#

A Collection is a class used to store and manage a group of objects dynamically. Unlike arrays, collections can grow or shrink during program execution.

Need for Collections

- Dynamic size
- Easy insertion and deletion
- Built-in searching and sorting
- Better data management

Types of Collections

Collection	Description	Example
ArrayList	Stores any type of object (non-generic)	Student names
Hashtable	Stores key-value pairs	Student Roll → Name
Stack	Last In First Out (LIFO)	Undo operation
Queue	First In First Out (FIFO)	Printer queue
Dictionary	Generic key-value collection	Employee ID → Name
List<T>	Generic dynamic array	List of integers

Generic Collection Classes

Generic Collection

List<T>

Dictionary<TKey,TValue>

Queue<T>

Stack<T>

HashSet<T>

Collections (ArrayList)

Why do we need Collections?

Suppose we want to store an unknown number of student names.

- **Array** has a fixed size.
- **Collection** can grow or shrink dynamically.

Without Collection (Array)

</> C#

```
string[] students = new string[3];
```

```
students[0] = "Ram";  
students[1] = "Hari";  
students[2] = "Sita";
```

```
// Cannot add another student  
// students[3] = "Gita"; // Error
```

Why Collection?

- Size is dynamic.
- Easy to add or remove items.
- No need to decide the size beforehand.

Problem: Array size is fixed.

With Collection (ArrayList)

</> C#

```
using System;  
using System.Collections;  
  
class Program  
{  
    static void Main()  
    {  
        ArrayList students = new ArrayList();  
  
        students.Add("Ram");  
        students.Add("Hari");  
        students.Add("Sita");  
        students.Add("Gita"); // No problem  
  
        foreach (var s in students)  
        {  
            Console.WriteLine(s);  
        }  
    }  
}
```

Output

```
Ram  
Hari  
Sita  
Gita
```

Generics (List<T>)

Why do we need Generics?

`ArrayList` can store **any type** of data, which may lead to mistakes.

Problem with ArrayList

```
ArrayList numbers = new ArrayList();

numbers.Add(10);
numbers.Add(20);
numbers.Add("Hello"); // Allowed, but incorrect
```

Now the collection contains both **integers** and a **string**.

This can cause runtime errors when performing mathematical operations.

Solution: Generic List<T>

```
using System;
using System.Collections.Generic;

class Program
{
    static void Main()
    {
        List<int> numbers = new List<int>();

        numbers.Add(10);
        numbers.Add(20);
        // numbers.Add("Hello"); // Compile-time Error

        foreach (int n in numbers)
        {
            Console.WriteLine(n);
        }
    }
}
```

Output

10
20

Collections vs Generics (Quick Comparison)

Collections (`ArrayList`)

Generics (`List<T>`)

Dynamic size

Dynamic size

Stores any data type

Stores only one specified type

Less type-safe

Type-safe

Older approach

Modern and recommended approach

- **Collections** solve the problem of **fixed-size arrays**.
- **Generics** solve the problem of **type safety** in collections.

In modern C#, use generic collections like `List<T>` instead of `ArrayList` whenever possible.

File IO in C#

File I/O (Input/Output) is the process of reading data from a file and writing data to a file. It allows programs to store data permanently instead of keeping it only in memory.

Need of File I/O

- Save user data permanently.
- Read previously stored data.
- Generate reports and logs.
- Exchange data between applications.

Common File I/O Classes

Namespace:

```
</> C#
```

```
using System.IO;
```

Class	Purpose
File	Read/write entire files
FileInfo	File information (size, creation date, etc.)
StreamReader	Read text files
StreamWriter	Write text files
Directory	Create, delete, and manage folders

Example 1: Writing to a File

```
</> C#  
  
using System;  
using System.IO;  
  
class Program  
{  
    static void Main()  
    {  
        File.WriteAllText("student.txt", "Ram, Age:20");  
  
        Console.WriteLine("Data written successfully.");  
    }  
}
```

Output

Data written successfully.

The file **student.txt** will contain:

Ram, Age:20

Example 2: Reading from a File

```
</> C#  
  
using System;  
using System.IO;  
  
class Program  
{  
    static void Main()  
    {  
        string data = File.ReadAllText("student.txt");  
  
        Console.WriteLine(data);  
    }  
}
```

Output

Ram, Age:20

Example 3: Append Data

Instead of replacing the file, append adds data at the end.

```
</> C#  
  
using System;  
using System.IO;  
  
class Program  
{  
    static void Main()  
    {  
        File.AppendAllText("student.txt", "\nSita, Age:21");  
  
        Console.WriteLine("Data appended.");  
    }  
}
```

Now the file contains

```
Ram, Age:20  
Sita, Age:21
```

Exception Handling

```
using System;  
using System.IO;  
  
class Program  
{  
    static void Main()  
    {  
        try  
        {  
            string data = File.ReadAllText("student.txt");  
            Console.WriteLine(data);  
        }  
        catch (Exception ex)  
        {  
            Console.WriteLine(ex.Message);  
        }  
    }  
}
```


Exception Handling in C#

Exception Handling is a mechanism used to detect and handle runtime errors so that the program does not terminate unexpectedly.

Examples of runtime errors:

- Dividing by zero
- Accessing an invalid array index
- Opening a file that does not exist
- Invalid user input

C# provides the following keywords:

- `try` – Contains code that may cause an exception.
- `catch` – Handles the exception.
- `finally` – Executes whether an exception occurs or not.
- `throw` – Explicitly throws an exception.

```
try
{
    // Risky code
}
catch (Exception ex)
{
    Console.WriteLine(ex.Message);
}
finally
{
    // Always executes
}
```

```
try
{
    int[] arr = {10,20};

    Console.WriteLine(arr[5]);
}
catch (DivideByZeroException)
{
    Console.WriteLine("Divide by zero.");
}
catch (IndexOutOfRangeException)
{
    Console.WriteLine("Array index is invalid.");
}
catch (Exception)
{
    Console.WriteLine("Some other exception occurred.");
}
```

Output

Array index is invalid.

Custom Exception in C#

Sometimes built-in exceptions are not enough.

Suppose a college allows admission only if age is 18 or above.

If the age is below 18, we want to throw our own exception.

This is called a Custom Exception.

```
using System;

class InvalidAgeException : Exception
{
    public InvalidAgeException(string message)
        : base(message)
    {
    }
}

class Program
{
    static void CheckAge(int age)
    {
        if (age < 18)
        {
            throw new InvalidAgeException("Age must be at least 18.");
        }

        Console.WriteLine("Admission Successful.");
    }
}
```

```
static void Main()
{
    try
    {
        CheckAge(16);
    }
    catch (InvalidAgeException ex)
    {
        Console.WriteLine(ex.Message);
    }
}
```

If we call:

`</> C#`

`CheckAge(20);`

Output

Age must be at least 18.

Output

Admission Successful.

LINQ (Language Integrated Query)

Fundamentals: Lambda Expressions

Syntax:

```
</> C#  
  
(parameter) => expression
```

or

```
</> C#  
  
(parameter) =>  
{  
    // statements  
}
```

Here, `=>` is called the **lambda operator** ("goes to").

A Lambda Expression is a short, anonymous function used in C#. It is commonly used with LINQ to filter, sort, search, and manipulate collections.

LINQ is a C# feature that simplifies querying, filtering, sorting, and transforming data from different data sources.

Without Lambda expressions, you would have to write separate methods for simple operations.

Without Lambda

```
</> C#  
  
using System;  
  
class Program  
{  
    static bool IsEven(int n)  
    {  
        return n % 2 == 0;  
    }  
  
    static void Main()  
    {  
        Console.WriteLine(IsEven(8));  
    }  
}
```

With Lambda

```
</> C#  
  
Func<int, bool> IsEven = n => n % 2 == 0;  
  
Console.WriteLine(IsEven(8));
```

Output

True

The lambda expression does the same work in a much shorter form.

Filtering Data using LINQ

Suppose we have a list of numbers and want only the even numbers.

```
using System;
using System.Linq;

class Program
{
    static void Main()
    {
        int[] numbers = { 2, 5, 8, 11, 14, 17, 20 };

        var evenNumbers = numbers.Where(n => n % 2 == 0);

        Console.WriteLine("Even Numbers:");

        foreach (var num in evenNumbers)
        {
            Console.WriteLine(num);
        }
    }
}
```

Explanation

- `Where()` filters data.
- `n => n % 2 == 0`
 - `n` is each number.
 - `% 2 == 0` checks if it is even.

Output

Even Numbers:

2

8

14

20

Finding Students with Marks > 60

```
using System;
using System.Linq;

class Program
{
    static void Main()
    {
        int[] marks = { 45, 78, 60, 82, 39, 95 };

        var passed = marks.Where(m => m > 60);

        Console.WriteLine("Students scoring above 60:");

        foreach (var m in passed)
        {
            Console.WriteLine(m);
        }
    }
}
```

Output

```
Students scoring above 60:
78
82
95
```

Sorting using Lambda

```
using System;
using System.Linq;

class Program
{
    static void Main()
    {
        string[] names =
        {
            "Ram",
            "Hari",
            "Sita",
            "Anita"
        };

        var sorted = names.OrderBy(name => name);

        foreach (var n in sorted)
        {
            Console.WriteLine(n);
        }
    }
}
```

Output

```
Anita
Hari
Ram
Sita
```


Common LINQ Methods with Lambda

LINQ Method	Purpose	Example
<code>Where()</code>	Filters data	<code>numbers.Where(n => n > 10)</code>
<code>Select()</code>	Projects/transforms data	<code>names.Select(n => n.ToUpper())</code>
<code>OrderBy()</code>	Sorts ascending	<code>students.OrderBy(s => s.Name)</code>
<code>OrderByDescending()</code>	Sorts descending	<code>students.OrderByDescending(s => s.Marks)</code>
<code>First()</code>	Returns first element	<code>numbers.First(n => n > 5)</code>
<code>FirstOrDefault()</code>	Returns first or default if none	<code>students.FirstOrDefault(s => s.Name=="Ram")</code>
<code>Count()</code>	Counts matching elements	<code>numbers.Count(n => n > 50)</code>
<code>Any()</code>	Checks if any element matches	<code>numbers.Any(n => n < 0)</code>

Lambda Expression Syntax Examples

Lambda Expression	Meaning
<code>x => x * 2</code>	Multiply by 2
<code>x => x % 2 == 0</code>	Check even number
<code>x => x > 100</code>	Check greater than 100
<code>(x, y) => x + y</code>	Add two numbers
<code>s => s.Name</code>	Return student's name
<code>s => s.Marks >= 60</code>	Students who passed

Why are Lambda Expressions Important in LINQ?

- ✓ Makes code **shorter** and **cleaner**.
- ✓ Eliminates the need to write separate methods for simple operations.
- ✓ Works seamlessly with LINQ methods like `Where()`, `Select()`, `OrderBy()`, and `Count()`
- ✓ Improves readability and maintainability.
- ✓ Widely used in modern C# and .NET applications.

Attributes in C#

Attributes are special tags in C# that provide additional information (metadata) about classes, methods, properties, fields, parameters, etc.

- *They do not directly affect program execution.*
- *They are mainly used by the compiler, .NET runtime, or reflection.*

Syntax:

```
</> C#  
  
[AttributeName]
```

or

```
</> C#  
  
[AttributeName(parameters)]
```

Attribute Classes

An **Attribute Class** is a class that inherits from the `Attribute` class.

You can use built-in attributes or create your own.

```
using System;

[Obsolete("Use NewMethod() instead.")]
class Demo
{
    public void OldMethod()
    {
        Console.WriteLine("Old Method");
    }

    public void NewMethod()
    {
        Console.WriteLine("New Method");
    }
}

class Program
{
    static void Main()
    {
        Demo d = new Demo();
        d.OldMethod();
    }
}
```

Example 1: Built-in Attribute

Output

Old Method

Compiler Warning

'OldMethod()' is obsolete: 'Use NewMethod() instead.'

Explanation

`[Obsolete]` tells the compiler that the method should no longer be used.

Example 2: Creating Your Own Attribute Class

```
using System;

class AuthorAttribute : Attribute
{
    public string Name;

    public AuthorAttribute(string name)
    {
        Name = name;
    }
}

[Author("Yuba Raj")]
class Book
{
}
```

```
class Program
{
    static void Main()
    {
        Console.WriteLine("Custom Attribute Example");
    }
}
```

Output

Custom Attribute Example

Here,

- `AuthorAttribute` inherits from `Attribute`
- It stores the author's name.

Named and Positional Attribute Parameters

Attributes can receive parameters in **two** ways.

A. Positional Parameters

- Passed through the constructor.
- Must come first.

Example

```
</> C#  
  
using System;  
  
[Obsolete("Use NewMethod")]  
class Demo  
{  
}
```

is a positional parameter because it is passed to the constructor.

Here,

"Use NewMethod"

B. Named Parameters

Named parameters assign values to public properties or fields.

Example

```
</> C#  
  
using System;  
  
class AuthorAttribute : Attribute  
{  
    public string Name;  
  
    public string Version { get; set; }  
  
    public AuthorAttribute(string name)  
    {  
        Name = name;  
    }  
}
```

```
[Author("Yuba Raj", Version = "1.0")]  
class Book  
{  
}  
  
class Program  
{  
    static void Main()  
    {  
        Console.WriteLine("Named Parameter Example");  
    }  
}
```

Output

Named Parameter Example

Here,

- "Yuba Raj" → Positional parameter
- Version = "1.0" → Named parameter

Attribute Targets

Attributes can be applied to different program elements called **targets**.

Common targets are:

- assembly
- class
- method
- property
- field
- parameter
- return value

Example 1: Class Target

```
</> C#  
  
using System;  
  
[Serializable]  
class Student  
{  
}
```

`Serializable` is applied to the **class**.

Example 2: Method Target

```
</> C#  
  
using System;  
  
class Demo  
{  
    [Obsolete]  
    public void Display()  
    {  
        Console.WriteLine("Display");  
    }  
}
```

The attribute applies to the **method**.

Example 3: Assembly Target

```
</> C#  
  
using System.Reflection;  
  
[assembly: AssemblyTitle("College Project")]  
  
class Program  
{  
    static void Main()  
    {  
    }  
}
```

The attribute applies to the **entire assembly**.

Example 4: Parameter Target

```
</> C#  
  
using System;  
  
class Demo  
{  
    public void Show([System.Runtime.InteropServices.Optional] int x)  
    {  
        Console.WriteLine(x);  
    }  
}
```

The attribute is attached to the parameter `x`.

Specifying Multiple Attributes

More than one attribute can be applied to the same element.

There are **two** ways.

Method 1: Separate Lines

```
</> C#  
  
using System;  
  
[Serializable]  
[Obsolete]  
class Demo  
{  
}
```

Method 2: Same Line

```
</> C#  
  
using System;  
  
[Serializable, Obsolete]  
class Demo  
{  
}
```

Both are equivalent.

```
using System;  
  
[Serializable]  
[Obsolete("Old Class")]  
class Student  
{  
}  
  
class Program  
{  
    static void Main()  
    {  
        Student s = new Student();  
  
        Console.WriteLine("Object Created");  
    }  
}
```

Output

Object Created

Compiler Warning

'Student' is obsolete: 'Old Class'

Summary Table

Topic	Description	Example
Attribute Classes	Custom classes derived from <code>Attribute</code>	<code>AuthorAttribute</code>
Positional Parameters	Constructor arguments	<code>[Author("Yuba Raj")]</code>
Named Parameters	Property/field assignments	<code>[Author("Yuba Raj", Version="1.0")]</code>
Attribute Targets	Apply attributes to assembly, class, method, property, parameter, etc.	<code>[Serializable]</code> , <code>[Obsolete]</code>
Multiple Attributes	Apply more than one attribute to the same element	<code>[Serializable][Obsolete]</code> or <code>[Serializable, Obsolete]</code>

- **Attribute:** A mechanism to add metadata to program elements.
- **Attribute Class:** A class derived from `System.Attribute` .
- **Positional Parameter:** Passed through the attribute constructor.
- **Named Parameter:** Assigned to a public property or field using `name = value` .
- **Attribute Targets:** Specify where an attribute is applied (class, method, property, parameter, assembly, etc.).
- **Multiple Attributes:** Multiple attributes can be applied using separate attribute declarations or a comma-separated list in a single declaration.

Asynchronous Programming in C#

Asynchronous programming allows a program to perform long-running tasks without blocking the main thread. Instead of waiting for one task to finish, the program can continue executing other tasks.

It is commonly used for:

- *File I/O*
- *Database operations*
- *Web API calls*
- *Network communication*

Synchronous vs Asynchronous

Synchronous

Executes one task at a time

Blocks the current thread

Program may freeze during long operations

Asynchronous

Can continue other work while waiting

Does not block the current thread

Program remains responsive

Principle of Asynchrony

The basic principle is:

Start a time-consuming task → Don't wait → Continue other work → Resume when the task finishes.

Real-life Example

Suppose you order food at a restaurant.

Synchronous

1. Order food.
2. Stand at the counter until food is ready.
3. Eat.

You waste time waiting.

Asynchronous

1. Order food.
2. Sit with friends and chat.
3. Waiter calls your number.
4. Eat.

You use waiting time productively.

Async and Await in C#

C# provides two keywords:

async

Marks a method as asynchronous.

`</> C#`

```
async Task MethodName()  
{  
}
```

Await

Pauses the async method without blocking the thread until the task completes.

```
await SomeTask();
```

```
using System;
using System.Threading.Tasks;
```

```
class Program
{
    static async Task Main()
    {
        Console.WriteLine("Program Started");

        await LongTask();

        Console.WriteLine("Program Finished");
    }

    static async Task LongTask()
    {
        Console.WriteLine("Task Started");

        await Task.Delay(3000); // Wait for 3 seconds

        Console.WriteLine("Task Completed");
    }
}
```

Output

```
Program Started
Task Started
(Waits 3 seconds)
Task Completed
Program Finished
```

Returning a Value

An async method can return data using `Task<T>`.

</> C#

```
using System;
using System.Threading.Tasks;

class Program
{
    static async Task Main()
    {
        int result = await AddNumbers();

        Console.WriteLine("Sum = " + result);
    }

    static async Task<int> AddNumbers()
    {
        await Task.Delay(2000);

        return 50 + 20;
    }
}
```

Output

```
(Waits 2 seconds)

Sum = 70
```


How async/await Works

```
Main()
|
|
Calls LongTask()
|
|
Task starts
|
await Task.Delay()
|
(Main thread is free)
|
Task finishes
|
Returns back
|
Program continues
```

Return Types in Async Methods

Return Type	Meaning
<code>Task</code>	No value is returned
<code>Task<T></code>	Returns a value of type <code>T</code>
<code>void</code>	Used mainly for event handlers

Advantages of Async Programming

- ✓ Keeps applications responsive.
- ✓ Better user experience.
- ✓ Efficient use of CPU.
- ✓ Handles multiple I/O operations efficiently.
- ✓ Improves scalability in web applications.

Limitations

- Code is slightly more complex.
- Harder to debug.
- Not useful for CPU-intensive tasks alone (consider parallel programming instead).

Old Questions 2078 – 2081

- Create a class named EMPLOYEE as super class and ENGINEER and DOCTOR as sub class. Make your own assumptions as properties and methods.
- What is the difference between abstract class, interface, and sealed class? When would you use each? Write a program demonstrating the use of abstract class with abstract methods and sealed class to prevent inheritance.
- What are indexers in C#? How are they different from properties? Explain with an example.
- Explain the async/await pattern in C#. Why is asynchronous programming important in web applications? Provide a simple example.
- Write short notes on :
 - a. File I/O operations in C# b. Method hiding vs Method overriding c. Lambda Expressions
- What are the needs for partial class and sealed class? How do you relate delegate with events?
- Differentiate between generic and non-generic collections. Write a simple program to create generic class with generic constructor, generic member variable, generic property and generic method.
- Differentiate between struct and enum. Why do we need to handle the exception? Illustrate with an example with your own customized exception.
- What is named and positional attribute parameters? Describe the .Net architecture design and principles.